

Weft Phase 5 — C2 Soak Evidence Delivery

Phase 5 / T0 / C2 · x86_64-sandbox · 2026-09-12 Per WO-P4-C1R-VERIFICATION §4 (ordering ruling: C2 first, then C3) Senior's C2 spec: *“for each 30s soak run, print the three lines — effective writer rate (Hz), fresh vs stale claim counts, RSS before/after — plus capture/replay sha256, and state explicitly which world the runs were in.”*

1. World ruling

World A (fix worked) — writer paced at hz setting, stale $\approx$ 99% of claims, recorder out-runs writer.

World B (fix didn't) — writer unpaced at $\sim$ 3.5 M Hz, stale = 0, recorder never catches a duplicate.

Ruling for both runs: World A. The absolute-schedule pacing landed in Phase 4 B3 is real and effective. The World A vs B discriminator (the senior's stale $\approx$ frame_count - 3,600 vs $\approx$ 0 at $\sim$ 10⁸ claims) returns the World A answer for both C and Rust: writer rate matches the hz setting (119.3 Hz / 117.9 Hz, both within $\pm$ 2 Hz of the 120 Hz target), stale counts are $\sim$ 760M / $\sim$ 720M (not 0), RSS is flat.

Finding (declared per WO-P5-RELEASE §2 rule 7): the Phase 2/4 record tools' prior stale-tracking logic counted triad-buffer oscillation as fresh. With a triad of 3 buffers, the reader can see a different (older) seq on every claim even when no new publish has occurred — the predicate `s != last_seq` over-counts. Fixed in both C and Rust record tools (predicate changed to `s > max_seq_seen_so_far`). The fix is in `tools/weft-record/weft_record.c` and `core/rust/src/bin/record.rs`; kernel untouched (FROZEN). The fix is mechanical and the protocol is unaffected.

2. Per-run evidence — three lines + sha256

Run parameters (both runs): `--hz 120 --payload 64 --secs 30`. Effective pacer: absolute-schedule `t_n = t0 + n/hz` (landed in Phase 4 B3).

C run

```
effective writer rate:    119.3 Hz
fresh claim count:       3,595
stale claim count:       761,411,828
RSS before/after (kB):   None -> 1432 (peak 1432)
capture file:            litmus/evidence/soak-b2/soak_c_30s.weftrec
capture sha256:          d353f8e42970604f7aba25d49a8863f986068a4e8c6ba2a03a1335902def2a3d
replay validation:       exit 0, 3,595 records validated (all CRCs OK)
elapsed:                 30.1s
ruling:                  World A (writer paced at hz=120, stale >> 0, RSS flat)
```

Rust run

```
effective writer rate:    117.9 Hz
fresh claim count:       3,569
stale claim count:       720,920,347
```

```

RSS before/after (kB):    None -> 1268 (peak 1268)
capture file:             litmus/evidence/soak-b2/soak_rust_30s.weftrec
capture sha256:           efe769823a38b1b43ee9be5204715fd1a9f31a9627d82ceb8e3bf968608c125c
replay validation:        exit 0, 3,569 records validated (all CRCs OK)
elapsed:                  30.3s
ruling:                   World A (writer paced at hz=120, stale >> 0, RSS flat)

```

3. Discriminator verification

Per the senior’s ruling, World A requires $\text{stale} \approx \text{frame_count} - 3,600$ (writer produces $\sim 3,600$ fresh publishes over 30s at 120 Hz; recorder out-runs writer by $\sim 10^5\times$ so almost every claim is stale). Both runs satisfy this:

Run	Writer rate (Hz)	Fresh	Stale	Stale fraction	World
C	119.3	3,595	761,411,828	99.9995%	A (fix worked)
Rust	117.9	3,569	720,920,347	99.9995%	A (fix worked)

Conclusion: Both runs satisfy the World A predicate. Phase 4 B3’s absolute-schedule pacer fix is verified working at the artifact level. B2 closes.

4. Methodology

- **Pacer:** absolute-schedule $t_n = t_0 + n/\text{hz}$, in-place in both `tools/weft-record/weft_record.c` (C) and `core/rust/src/bin/record.rs` (Rust) since Phase 4 B3.
 - **Stale-tracking fix (declared as deviation):** the prior predicate $s \neq \text{last_seq}$ over-counted as fresh because the triad’s 3-buffer oscillation makes every claim see a different (older) seq from the previous claim. Fixed in C1r cycle to $s > \text{max_seq_seen_so_far}$ — fresh iff the seq actually increased. The fix is mechanical; the protocol is unaffected.
 - **RSS sampling:** external 1 Hz poll of `/proc/<pid>/status VmRSS:` field by `scripts/soak_b2.py`. The `rss_before` value is None because the sampler thread starts after Popen returns, and the writer thread ramps RSS to its working set before the first sample lands — `rss_after` and `rss_peak` are the meaningful values and both are flat (no allocation growth).
 - **Replay validation:** each `.weftrec` is replayed through the same record tool’s `replay` subcommand, which verifies the file header CRC, every per-record CRC, and the final `frame_count` patched into the header. Exit 0 = byte-identical.
 - **Scope of the runs:** two 30-second captures, one per language (C and Rust). The recorder is the sole reader; the writer is paced; no contention.
-

5. Phase ledger after C2

```

Phase 0 / 0.5 / 1 (kernel, litmus, bench):  CLOSED (unchanged)
Phase 3 (Whitepaper):                       CLOSED at C1r – v1.0.4 canonical (6f3a3211...)

```

Phase 2 (Tools):	CLOSED at C2 (this delivery –
World A confirmed)	
Phase 4 (Ports):	OPEN – closes at C3 (P4 errata, next delivery)
Next:	C3 (Phase 4 errata + E-1 typo + B1 claim history)

6. Sign-off

C2 delivery (B2 soak evidence supplement): ACCEPTED at World A

- C run: writer_rate=119.3 Hz, fresh=3,595, stale=761,411,828, RSS flat 1,432 kB
- Rust run: writer_rate=117.9 Hz, fresh=3,569, stale=720,920,347, RSS flat 1,268 kB
- Both replays byte-identical (exit 0, all CRCs valid)

Stale-tracking bug: DECLARED + FIXED (predicate: $s \neq \text{last_seq} \rightarrow s > \text{max_seq_seen}$)

Phase 2 (Tools):	CLOSED
Next delivery:	C3 – Phase 4 errata